

Pipeline-ul OpenGL programabil - Fragment shader

În Figura 1 este reprezentat pipeline-ul caracteristic platformelor ce implementează versiuni noi ale OpenGL. Cu albastru sunt desenate etapele ce pot fi programate.

Nota: între vertex shader și geometry shader mai există un pas opțional ce poate fi programat, numit tessellation shader.

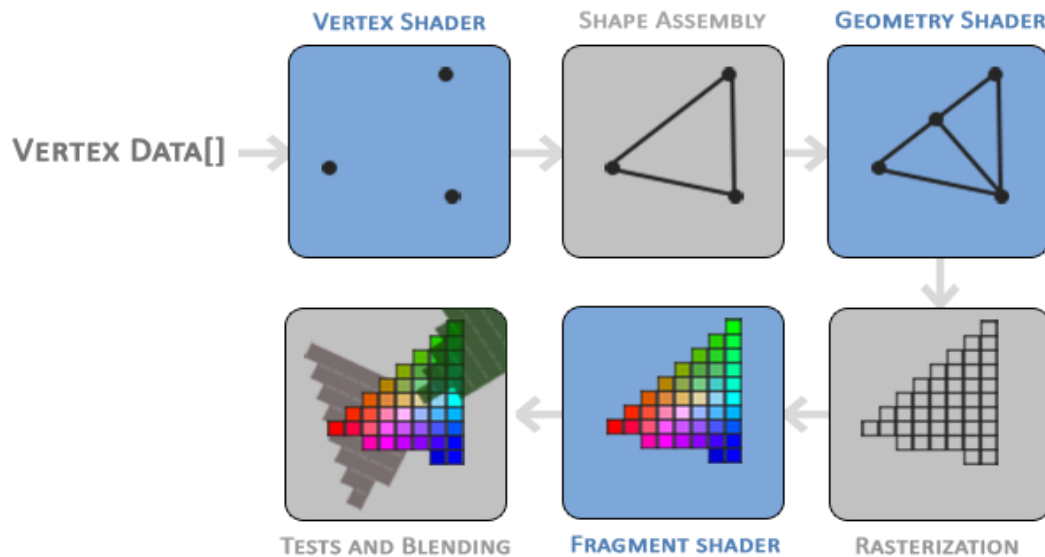


Figura 1. Pipeline-ul programabil OpenGL

1. Fragment shader

Ultimul pas programabil al pipeline-ului OpenGL este reprezentat de către **fragment shader**. Acest shader este rulat pentru fiecare pixel care va fi colorat pentru a-i atribui o culoare. Shaderul fragment trebuie să aibă o singură variabilă de ieșire de tipul `vec4`. Această variabilă va fi culoarea pixelului.

De exemplu, programul fragment shader care a fost utilizat la laboratoarele precedente a avut următorul conținut (toți pixelii aparținând obiectului desenat vor avea aceeași culoare specificată în shader):

```
#version 400
out vec4 frag_colour;
void main() {
    frag_colour = vec4(0.5, 0.0, 0.5, 1.0);
}
```

Această culoare se poate atribui direct, ca în exemplul precedent, sau poate fi transferată către fragment shader. O metodă de a transfera o culoare către fragment shader este aceea de a transfera un

vector cu 4 elemente din C++ către shader utilizând variabile `uniform` (vezi laboratoarele precedente, când s-au desenat diferite obiecte cu culori diferite).

O altă metodă de a transfera informații către fragment shader este aceea de a utiliza transferul de informații între shadere. Acest lucru înseamnă că shaderul vertex poate transmite informații către shaderul fragment. Aceste informații pot include culoarea pixelului sau alte informații necesare la calcularea culorii pixelului.

Un exemplu de transmitere a informațiilor de la vertex shader la fragment shader este:

Shaderul vertex:

```
#version 400
in vec3 aPos;
out vec4 vertexColor;

void main()
{
    gl_Position = vec4(aPos, 1.0);
    vertexColor = vec4(0.5, 0.0, 0.0, 1.0);
}
```

Shaderul fragment:

```
#version 400
out vec4 fragColor;
in vec4 vertexColor;

void main()
{
    fragColor = vertexColor;
}
```

Și în acest exemplu toți pixelii triunghiului vor avea aceeași culoare, dar această culoare este transmisă de către shaderul vertex și folosită apoi în shaderul fragment. Pentru a se putea realiza acest transfer, ambele variabile, cea care transmite informațiile din shaderul vertex și cea care primește informațiile în shaderul fragment trebuie să aibă același nume și același tip.

2. Transmiterea culorii prin intermediul Vertex Buffer Object

În laboratoarele precedente, am transmis către vertex shader pozițiile primitivelor geometrice pe care doream să le desenăm. Pe lângă pozițiile acestor vârfuri, putem transmite cu ajutorul aceluiași VBO și alte informații cu privire la primitivele geometrice, precum culoarea. Pentru a transmite și culoarea, adăugăm aceste informații la vectorul de poziții deja existent:

```
float points[] = {
    //pozitii          culoare
    0.0f,  0.5f,  0.0f,  1.0f,  0.0f,  0.0f,
    0.5f,  -0.5f,  0.0f,  0.0f,  1.0f,  0.0f,
    -0.5f,  -0.5f,  0.0f,  0.0f,  0.0f,  1.0f
};
```

Pentru că am adăugat informații la vectorul de puncte existent, trebuie să modificăm și shaderul vertex pentru a putea primi și aceste informații de culoare. Shaderul vertex va fi:

```
#version 400
//the position variable has attribute location 0:
layout (location = 0) in vec3 aPos;
//the color variable has attribute location 1:
layout (location = 1) in vec3 aColor;
out vec3 vertexColor; // outputs a color to the fragment shader

void main()
{
    gl_Position = vec4(aPos, 1.0);
    vertexColor = aColor;
}
```

Shaderul fragment va avea următorul conținut:

```
#version 400
out vec4 FragColor;
in vec3 vertexColor;

void main()
{
    FragColor = vec4(vertexColor,1.0);
}
```

Pentru că am adăugat atribute noi pentru vertex-urile existente trebuie să reconfigurăm obiectele vertex array folosite. Datele din VBO vor arata ca în figura 2:

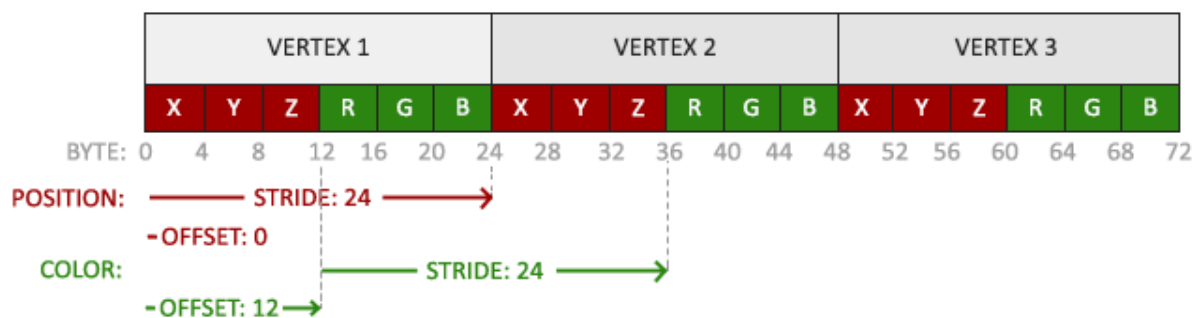


Figura 2. Modul de aranjare a datelor în VBO-ul folosit în exemplul dat

Cunoscând acest mod de aranjare a datelor în memorie, trebuie să modificăm formatul datelor vertex attribute. Acest lucru se face utilizând următoarele instrucțiuni:

```
// position attribute
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE,
                     6 * sizeof(float), NULL);
glEnableVertexAttribArray(0);
// color attribute
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE,
                     6 * sizeof(float), (void*)(3 * sizeof(float)));
glEnableVertexAttribArray(1);
```

Primul argument al funcției `glVertexAttribPointer` specifică indexul atributului vertex ce va fi modificat. Acest index trebuie ulterior activat cu funcția `glEnableVertexAttribArray`. Tot acest index trebuie să corespundă cu valorii specificate pentru `location` în shaderul vertex. În exemplul prezentat, cu indexul 0 se transmit pozițiile vertexurilor, iar cu indexul 1 se transmit culorile. Pentru că acum avem mai multe date în VBO, trebuie să calculăm parametrul `stride` pentru fiecare atribut vertex (poziție, respectiv culoare). Acest parametru specifică offset-ul în octeți dintre două elemente ce reprezintă aceleași proprietăți ale vertexurilor consecutive. Pentru poziții, specifică numărul de octeți dintre pozițiile a două vertexuri consecutive. În exemplu prezentat, parametrul `stride` trebuie să fie `6 * sizeof(float)` deoarece între două elemente de tip poziție există 6 valori de tip `float`.

De asemenea, pentru atributul culoare, trebuie specificat un offset față de poziția de început a datelor de culoare față de începutul array-ului ce conține informațiile despre vertexuri. În cazul de față, trebuie să specificăm un offset de 3 elemente `float`. Cu alte cuvinte, prima culoare se va afla în memorie după 3 elemente `float` ce reprezintă poziția (coordonatele x, y, z) primului punct.

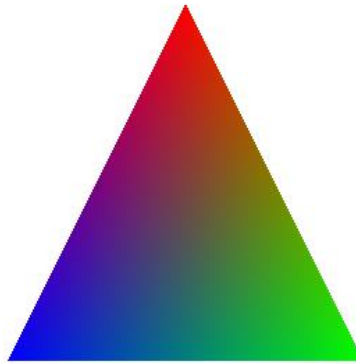


Figura 3. Rezultatul rulării programului cu transmiterea culorilor pentru fiecare vârf al triunghiului

Pentru a obține imaginea din Figura 3 s-au transmis doar culorile vârfurilor triunghiului. Totuși, triunghiul este colorat cu foarte multe culori. Acest lucru se întâmplă datorită procedurii numite interpolarea fragmentelor, procedeu ce are loc în shaderul fragment. La randarea unui triunghi, în cadrul procesului de rasterizare vor rezulta mult mai multe fragmente decât vertex-urile specificate de utilizator. Procesul de rasterizare va determina poziția acestor fragmente în funcție de unde se află ele în interiorul triunghiului. Pe baza acestor poziții se vor interpola toate variabilele de intrare pentru shaderul fragment.

Exercițiu

Completați exemplul furnizat în directorul laboratorului, în arhiva *SPG_lab7_triunghi.zip* (problema care realizează desenarea unui triunghi pe ecran), pentru a transmite către vertex shader și informația de culoare pentru fiecare vertex în afară de poziția sa. Apoi informația de culoare trebuie să fie transmisă din vertex shader în fragment shader. Trebuie să obțineți imaginea din Figura 3.

Probleme propuse

1. Modificați programul anterior astfel încât să se transmită în același VBO 2 triunghiuri care formează un patrat. Fiecare vertex va avea asociată și o culoare (R,G,B) în VBO. Faceți modificările necesare pentru a transmite culoarea sub forma unui vector (R, G, B, A).
2. Să se deseneze triunghiul colorat jumătate cu o culoare jumătate cu alta. Indicație: trebuie să trimiteți de la vertex shader la fragment shader o informație în funcție de care să se aleagă culoarea cu care se va desena fragmentul respectiv.
3. Modificați exemplul din fișierul *SPG_lab7_sfera.zip* pentru a desena o sferă colorată jumătate de o culoare jumătate de o altă culoare, după modelul din figura următoare.

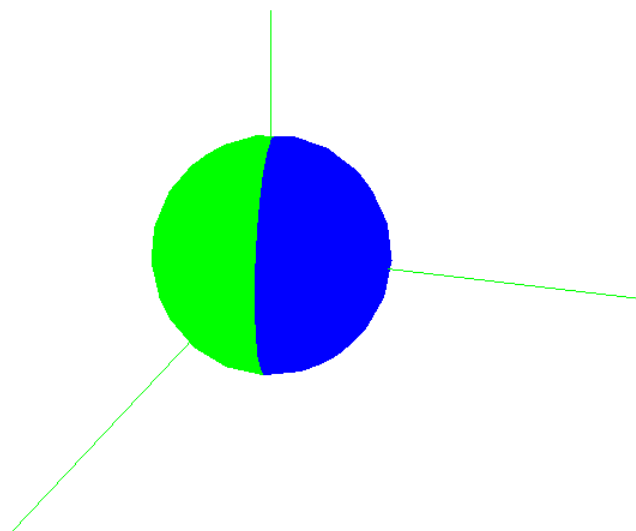


Figura 4. Sferă reprezentată folosind două culori

4. Determinați culoarea sferei funcție de unghiul pe care îl formează normala la suprafață cu axa OX, așa cum se ilustrează în Figura 5.

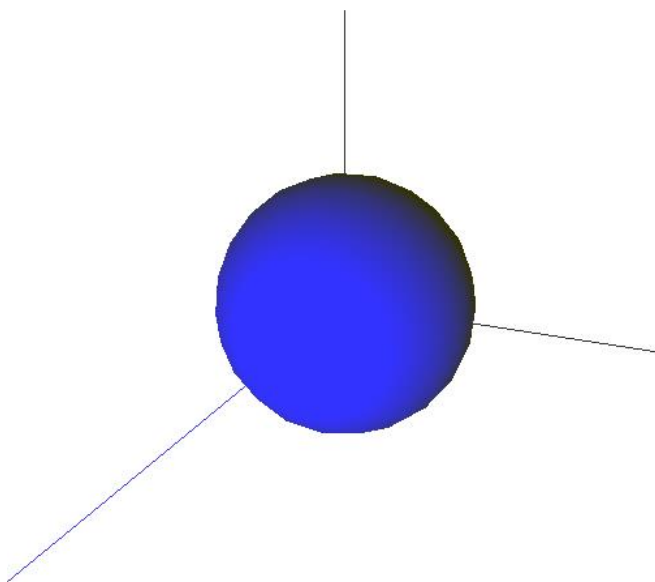


Figura 5. Sferă colorată funcție de orientarea normalei la suprafață

Normala într-un punct de pe o suprafață este vectorul perpendicular pe planul tangent la suprafață în acel punct (Figura 6). În cazul unei sfere centrate în origine, vectorii normali la suprafață sunt ușor de calculat, ei având direcția dată de poziția vertexurilor corespunzătoare. În Figura 7(a), se observă cum dreapta ce trece prin origine și prin poziția unui vertex are aceeași direcție ca normala la suprafață corespunzătoare acelui vertex.

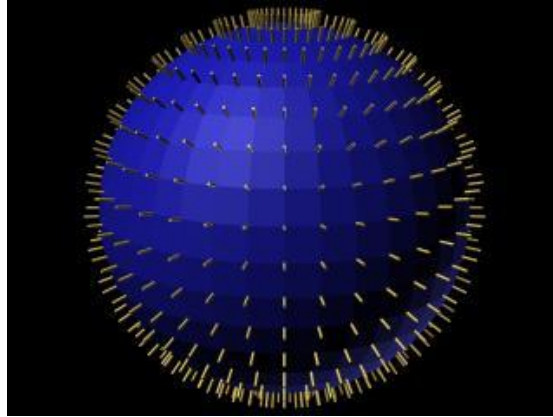


Figura 6. Reprezentarea normalelor la suprafață pentru fiecare vertex de pe suprafața unei sfere

Pentru a obține efectul din Figura 5, valoarea culorii stabilite pentru sferă se va înmulți cu cosinul unghiului dintre axa OX și normala la suprafață. Cu cât acest unghi este mai mic, cu atât cosinul său va fi mai mare și cu atât culoarea va fi mai pronunțată, iar cu cât acest unghi este mai mare, cu atât cosinul său va fi mai mic și cu atât culoarea se va atenua (Figura 7(b)).

Cosinul unghiului dintre normala la suprafață și axa OX se va determina folosind produsul scalar dintre vectorul normalei și un vector cu aceeași orientare ca și axa OX. Produsul scalar a doi vectori este prezentat în Ecuația 1. În cazul nostru, cosinul unghiului se determină ca fiind produsul scalar al celor doi vectori normalizați (Ecuația 2). (vector normalizat = un vector cu aceeași direcție și cu lungimea 1). Operațiile de normalizare și produs scalar sunt deja definite în glsl și glm. Pentru normalizare se poate folosi funcția **normalize** pentru vectori din GLSL, respectiv **glm::normalize** pentru vectori din GLM (de tipul glm::vec3), iar produsul scalar se numește *dot product* în engleză, așadar funcțiile corespunzătoare sunt **dot**, respectiv **glm::dot**.

$$\vec{V}_1 \cdot \vec{V}_2 = \|\vec{V}_1\| \|\vec{V}_2\| \cos(\widehat{V_1, V_2}) \quad (1)$$

$$\cos(\widehat{N, OX}) = \text{dot}(\text{normalize}(\vec{N}), \text{normalize}(\vec{OX})) \quad (2)$$

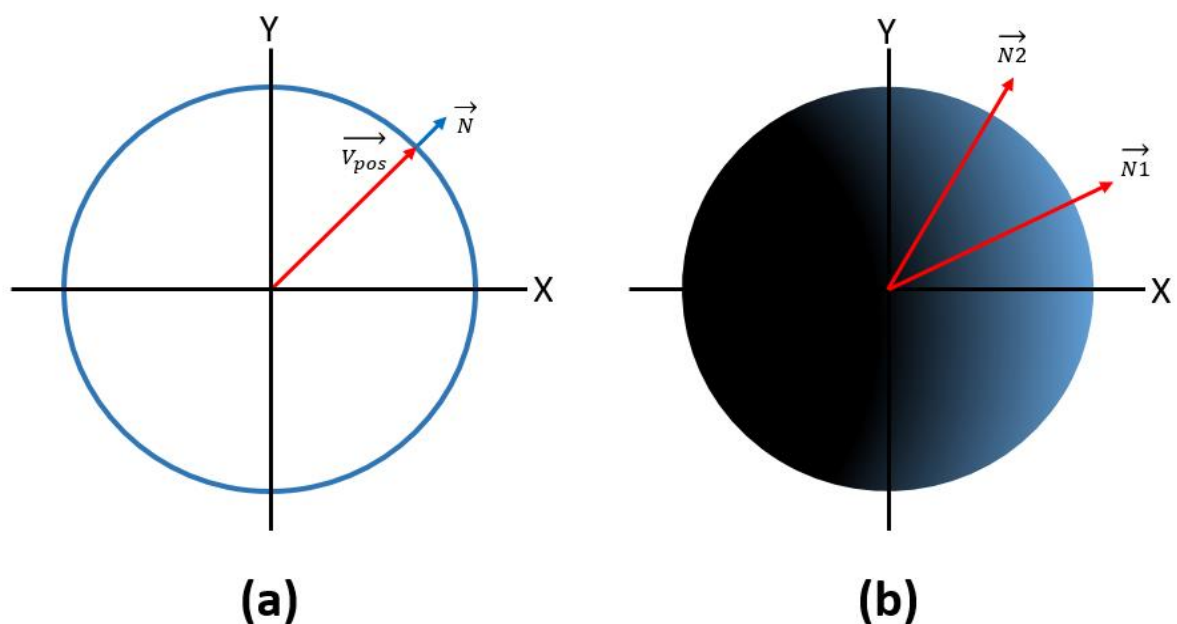


Figura 7. (a) Normala N la suprafața sferei centrate în origine are aceeași orientare ca și vectorul V_{pos} definit de poziția vertexului corespunzător; (b) Normalele $N1$ și $N2$ a două puncte de pe suprafața sferei formează unghiuri diferite cu axa OX . Culoarea sferei se poate ajusta funcție de valorarea acestui unghi. În cazul de față, cu cât unghiul este mai mic, cu atât culoarea este mai pronunțată.